UNIDAD DIDÁCTICA 4

BASES DE DATOS RELACIONALES Y ORIENTADAS A OBJETO

MÓDULO PROFESIONAL: ACCESO A DATOS

ÍNDICE

RESUMEN INTRODUCTORIO 2
INTRODUCCIÓN 2
CASO INTRODUCTORIO 2
1.CARACTERÍSTICAS DE LAS BASES DE DATOS OBJETO-RELACIONALES 4
1.1 Gestores de bases de datos objeto relacionales 6
1.1.1 Tipos de datos 6
1.1.2 PostgreSQL 8
1.1.3 Oracle 10
1.2 Gestión de objetos con SQL 11
1.3 La interfaz de programación de aplicaciones de la base de datos 13
1.3.1 Acceso a las funciones del gestor desde el lenguaje de programación 14
1.3.2 Acceso y uso de tipos especiales 16
1.4 Configuración de un Proyecto con PostgreSQL 16
1.5 Ventajas de los gestores de bases de datos objeto relacionales 26
2. CARACTERÍSTICAS DE LAS BASES DE DATOS ORIENTADAS A OBJETOS 28
2.1 Estructura de una BDOO 30
2.2 Gestores de bases de datos orientadas a objetos 33
2.2.1 ObjectStore 33
2.2.2 Wakanda 34
2.2.3 ObjectDB 35
2.3 Lenguaje de consultas para objetos (OQL, Object Query Language) 38
2.4 Gestión de la Persistencia y Transacciones. Consultas en Java con JPA 40
2.5 Configuración de un Proyecto con ObjectDB 42
2.6 Ventajas de las bases de datos orientadas a objetos 50
RESUMEN FINAL 52

RESUMEN INTRODUCTORIO

En esta unidad conoceremos los conceptos básicos sobre dos modelos de bases de datos y sistemas de gestión de bases de datos, bases de datos objeto-relacionales y las bases de datos orientadas a objetos.

En el primer caso, veremos cómo las bases de datos relacionales tradicionales han ido evolucionando, incorporando elementos y estructuras más complejas que nos permiten realizar una gestión más cercana a la programación orientada a objetos. Veremos algunas de estas soluciones y cómo el lenguaje SQL y el API JDBC de Java nos permite evolucionar nuestras aplicaciones.

En segundo lugar, estudiaremos las bases de datos orientadas a objetos, que heredan las características y arquitectura de la programación orientada a objetos y por lo tanto reducen el desfase objeto-relacional. Veremos también ejemplos comerciales y realizaremos algún ejemplo de uso.

INTRODUCCIÓN

Además de las soluciones de bases de datos relacionales, tenemos más alternativas que se pueden adaptar a las necesidades de negocio de una empresa y que intentan minimizar el desfase objeto-relacional.

La primera de las alternativas está protagonizada por los mismos SGBD relacionales. El fuerte arraigo del paradigma orientado a objetos en el diseño y la implementación de las aplicaciones actuales está obligando los SGBD relacionales a incorporar características orientadas a objetos en los elementos de diseño y explotación de los sistemas gestores. Es el que se ha bautizado como Sistemas Gestores de Bases de datos Objeto-Relacionales.

La segunda alternativa consiste en usar directamente un sistema gestor de bases de datos orientado a objetos. Por supuesto, se trata de la única alternativa que no presenta desfase Objeto-Relacional, puesto que los datos se tratan siempre, incluso durante el almacenamiento, como si fueran objetos y no tablas.

CASO INTRODUCTORIO

En la empresa que trabajas como desarrollador junior de aplicaciones, tu jefe de departamento se plantea realizar pruebas de concepto sobre diferentes bases de datos a la que actualmente se está usando, MySQL.

La empresa se dedica al estampado de ropa deportiva, y la aplicación gestiona múltiples aspectos dentro de la empresa, toda ella realizada con Java, y con MySQL como base de datos como hemos indicado.

Para la prueba de concepto, tu jefe se plantea realizar pruebas sobre el pedido desde cliente de una nueva equipación.

Al finalizar la unidad conocerás los distintos aspectos a considerar en cuanto a las aplicaciones que gestionan la información almacenada en bases de datos objeto relacionales y orientadas a objetos.

1.CARACTERÍSTICAS DE LAS BASES DE DATOS OBJETO-RELACIONALES

El jefe de departamento se plantea evaluar soluciones alternativas a la actual, comparar rendimientos, estudiar y medir las posibilidades de nuevos tipos de bases de datos.

Actualmente, en la empresa el almacenamiento se basa en el uso de MySQL, pero se plantea la opción de usar PostgreSQL, comprobar las herramientas que tiene de funciones, de conexión con los objetos y nuevos tipos. Con PostgreSQL podrás aprovechar el potencial de la orientación a objetos en una base de datos de naturaleza relacional.

El concepto de **base de datos objeto relacional (BDOR)** se utiliza para referirse a las bases de datos que han evolucionado desde el modelo relacional incorporando conceptos del paradigma orientado a objetos. Por tanto, las bases de datos objeto relacionales incorporan las características del modelo relacional y las del modelo de objetos.

La **principal característica** de estas bases de datos es que el usuario puede crear sus propios tipos de datos y los métodos necesarios para trabajar con dichos tipos de datos. De esta manera, se permite la creación de funciones miembro usando tipos de datos definidos por el usuario y, al mismo tiempo, proporciona mayor flexibilidad y seguridad.

Los **SGBDOR (Sistemas de gestión de bases de datos objeto relacional)** proporcionan, por tanto, grandes mejoras con respecto a los SGBDR tradicionales. Por ejemplo, como se ha comentado anteriormente, permiten gestionar tipos de datos complejos sin que esto conlleve un gran esfuerzo. Además, mantienen parte de la aplicación en el servidor de base de datos.

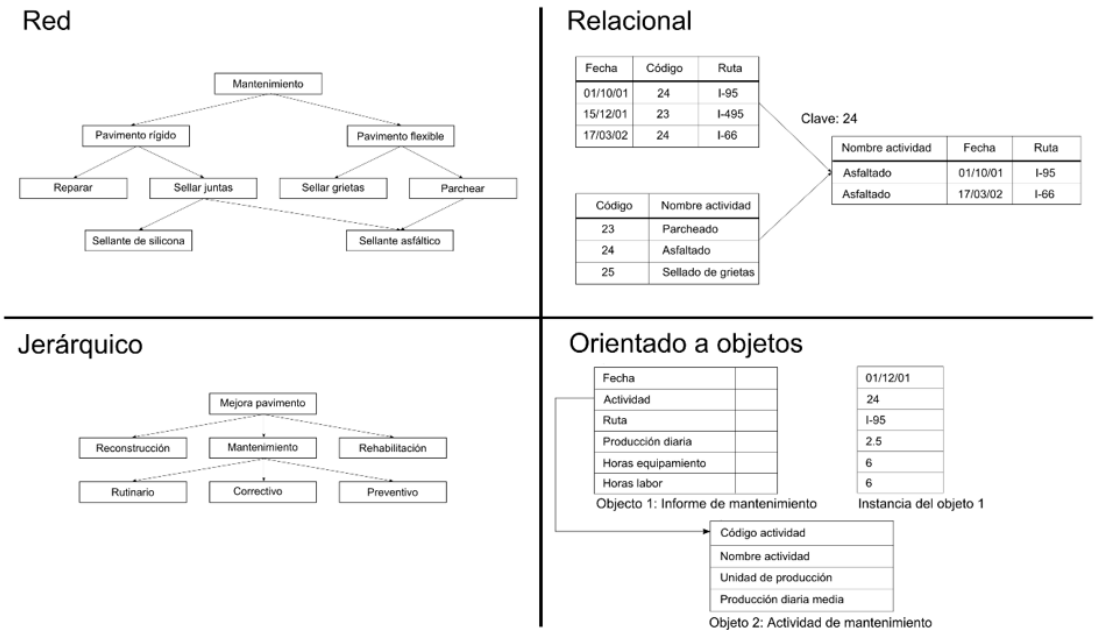
Otra característica que se puede destacar es el hecho de que permiten almacenar datos complejos de una aplicación en la base de datos objeto-relacional sin necesidad de forzar los tipos de datos tradicionales.

Los SGBDOR son **compatibles en sentido ascendente** con los SGBDR tradicionales, lo cual aporta una gran facilidad de aprendizaje y utilización puesto que a menudo el usuario está muy familiarizado con los SGBD relacionales. Además, esto permite pasar las aplicaciones sobre BDR al modelo objeto relacional sin necesidad de rescribirlas.

Debido a que las BDOR están cogiendo bastante fuerza, los SGBD se han visto obligados a incorporar conceptos del modelo orientado a objetos. De esta manera, los usuarios pueden usar los mecanismos de orientación a objetos para definir y acceder a los datos independientemente de que estos estén almacenados en forma de tablas.

Oracle, desde la **versión 8.i**, ha incorporado estos conceptos de orientación a objetos y, por tanto, proporciona mecanismos para que el usuario pueda definir sus **propios tipos de datos**, así como para **asociar métodos a tipos de datos**.

En la siguiente imagen se muestra un esquema comparando los SGBD tradicionales con los nuevos que incorporan conceptos de la orientación a objetos.



Comparativo de arquitectura de Sistemas de gestión de bases de datos
Fuente: Elaboración propia

BIBLIOGRAFÍA RECOMENDADA

Te recomendamos el siguiente libro para conocer más sobre los sistemas de bases de datos y donde encontrar más información sobre las bases de datos objeto-relacionales.

Begg, C.E., y Connolly, T.M. (2010). *Sistemas de bases de datos* (4.a ed.). Madrid: Pearson Addison Wesley.

VÍDEO DE INTERÉS

Amplia tus conocimientos sobre las bases de datos orientadas a objetos visualizando este vídeo.

1.1 Gestores de bases de datos objeto relacionales

Un **SGBD** representa un elemento intermedio entre la información o datos almacenados en las bases de datos y las aplicaciones que van a utilizar dichos datos. Su principal objetivo es facilitar las operaciones que se quieran realizar con la información.

En unidades anteriores, se estudiaron las principales características de estos, entre las que destacaban:

- Protección de los datos.
- Eficiencia.
- Gestión de transacciones.

Puesto que ya se estudió el concepto y características de éstos, este apartado se centrará en estudiar los **SGBD que incorporan el nuevo paradigma orientado a objetos**. Estos SGBD son una evolución de los SGBD relacionales, incorporando algunas características de la orientación a objetos, por lo que pasan a ser llamados como sistemas de gestión de bases de datos objeto-relacional (**SGBDOR**).

1.1.1 Tipos de datos

Los tipos de datos utilizados en los SGBD se pueden clasificar en:

- **Atómicos:**
  - **Boolean**: un valor booleano es aquel que puede tomar uno de los siguientes valores: verdadero o falso.
  - **Short:** entero con signo, normalmente de 8 o 16 bits.
  - **Long:** entero con signo, normalmente de 32 o 64 bits.

- **Unsigned short**: entero sin signo, normalmente de 8 o 16 bits.
- **Unsigned long**: entero sin signo, normalmente de 32 o 64 bits.
- **Float:** valor real en coma flotante de simple precisión.
- **Double:** valor real en coma flotante de doble precisión.
- **Octet:** almacén de 8 bits.
- **Char:** carácter ascii o unicode.
- **String:** cadena de caracteres.
- **Enum**: tipo enumerado donde los valores se especifican explícitamente cuando se declara el tipo.

- **Colecciones**. Se derivan de la interface Collection. Se pueden distinguir los siguientes:
  - **Set y bag**: grupo desordenado de objetos del mismo tipo. La diferencia está en que set no permite duplicados y bag sí.
  - **List:** grupo ordenado de objetos del mismo tipo.
  - **Array:** grupo ordenado de objetos del mismo tipo. Destaca porque se puede acceder a ellos mediante su posición.

- **Tipos estructurados**. Son los siguientes:
  - **Date:** para las fechas.
  - **Time:** para las horas.
  - **Timestamp:** hora de una fecha.
  - **Interval:** periodos de tiempo.

PARA SABER MÁS

En esta web podrás comprobar un resumen de los tipos de datos utilizados en el SGBD Oracle:

1.1.2 PostgreSQL

Sistema de gestión de bases de datos relacional **libre** que destaca por agilizar la interacción de cliente, servidor y base de datos, ya que realiza la mayoría del trabajo referente a bases de datos cuando se hacen las peticiones.

Entre sus principales virtudes se pueden destacar la estabilidad, potencia, robustez, facilidad de administración e implementación de estándares. Además, este SGBD funciona a la perfección con bases de datos que almacenan grandes cantidades de datos y una alta concurrencia de usuarios accediendo simultáneamente a la base.

Sus **principales características** son:

- Es un SGBD fácil de administrar.
- Facilidad de aprendizaje ya que usa sintaxis SQL estándar.
- Es multiplataforma.
- Permite la replicación de datos.
- Permite la realización de lo que se denomina copias de seguridad en caliente.
- Es un SGBDOR, por lo que entre sus mecanismos incluye herencia entre tablas, así como otros mecanismos orientados a objetos.
- Es multiplataforma, por lo que está disponible para Linux, Unix, Windows, etc, en todas sus distribuciones y versiones.
- Permite, gracias al sistema MVCC, que mientras un proceso escribe en una tabla, otros accedan a la misma tabla sin necesidad de bloqueos.
- Permite la creación de tipos de datos, además de soportar distintos tipos diferentes de los tipos base como, por ejemplo, fechas, monedas, cadenas de bits, etc.
- Soporta juegos de caracteres internacionales, así como Unicode.

En cuanto a sus **limitaciones o desventajas** cabría citar:

- Actualmente no permite la realización de puntos de recuperación dentro de las transacciones por lo que, si se encuentra un fallo durante su ejecución, las transacciones abortan completamente.
- Aunque incorpora mecanismos de herencia y algunos otros de la orientación a objetos, pero cabe decir que no ofrece completas prestaciones de orientación a objetos.
- No soporta tablespaces para definir dónde almacenar la base de datos, el esquema, los índices, etc.
- Es más lento en inserciones y actualizaciones que MySQL, ya que cuenta con cabeceras de transacciones (Transaction Headers) que no tiene MySQL.

- Aunque existen multitud de foros oficiales donde exponer dudas y cuestiones varias, no ofrece un sistema de ayuda.
- En comparación con otros SGBD como, por ejemplo, MySQL, consume más recursos.
- La sintaxis de algunos de sus comandos o sentencias no es nada intuitiva, lo cual dificulta su aprendizaje.

PARA SABER MÁS

En la web oficial de este SGBD se encuentra la documentación oficial, así como foros y otros sistemas de ayuda:

EJEMPLO PRÁCTICO

Trabajas en una empresa de estampado de camisetas. Tu jefe ha solicitado que se almacenen los datos de los clientes de manera que se incluyan las preferencias de estilo de ropa deportiva (como colores y tipos de prendas) en la base de datos, usando PostgreSQL. Te pide que lo crees a través de tipos personalizados dentro de la base de datos de PostgreSQL.

¿Cómo lo realizarías?

- Para resolver el problema necesitas definir el tipo personalizado.

```
CREATE TYPE preferencia_tipo AS (
    color VARCHAR(50),
    prenda VARCHAR(50)
);
```

- Crear la tabla clientes:

```
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    preferencias preferencia_tipo[]
);
```

- Una vez creado, ya puedes probarlo insertando clientes.

```
INSERT INTO clientes (nombre, preferencias)
VALUES ('Ana Lopez', ARRAY[('Rojo', 'Camiseta'), ('Azul',
'Pantalón')]);
```

1.1.3 Oracle

Sistema de gestión de bases de datos objeto-relacional fabricado por Oracle Corporation, que destaca en el mercado por su portabilidad y compatibilidad con distintos sistemas.

Además, entre las principales **características**, cabe destacar:

- Utiliza la arquitectura cliente servidor.
- Destaca por gestionar a la perfección bases de datos que almacenan grandes cantidades de datos y a las que acceden multitud de usuarios concurrentes.
- Proporciona un alto rendimiento en las transacciones.
- Sistemas de alta disponibilidad.
- Disponibilidad controlada de los datos de las aplicaciones.
- Adaptación a estándares de la industria, como SQL-92.
- Gestión de la seguridad.
- Autogestión de la integridad de los datos.
- Opción distribuida.
- Conectabilidad.
- Replicación de entornos.

Al ser un sistema propietario, como principal inconveniente cabe destacar su elevado precio, tanto al adquirirlo como en cuanto a soporte técnico y mantenimiento. Además, es necesario aplicar parches de seguridad debido a las vulnerabilidades en la seguridad de la plataforma.

ENLACE DE INTERÉS

Aquí conocerás cómo se estructura Oracle:

1.2 Gestión de objetos con SQL

SQL permite organizar, gestionar y recuperar datos almacenados en una base de datos informática. Es un lenguaje que se utiliza para interactuar con una base de datos y, de forma específica, con un tipo llamado base de datos relacional. En la actualidad es el lenguaje de uso y programación de bases de datos relacionales más extendido.

La **historia del lenguaje SQL** comienza en el año 1974 con la definición de un lenguaje para la especificación de las características de las bases de datos relacionales. Este lenguaje se llamaba SEQUEL y fue modificándose, dando lugar a nuevas revisiones de éste y llegando a cambiar su nombre por motivos legales convirtiéndose en el lenguaje SQL tal como se conoce hoy en día.

Otra versión destacable a lo largo de la historia sería la versión SQL-92, el cual fue desarrollado por el comité técnico NCITS sobre bases de datos. Este lenguaje fue creado como estándar para los sistemas de gestión de bases de datos relacionales.

A continuación, en 1999 fue desarrollado el lenguaje SQL 3, que venía a mejorar las anteriores versiones e incorporar la orientación a objetos. Por tanto, este lenguaje es considerado como la base de la mayoría de los SGBD entre los que se puede destacar, por ejemplo, Oracle.

El estándar **ANSI SQL** asegura que el lenguaje SQL mantenga una sintaxis y funcionalidad coherentes a través de diferentes sistemas de gestión de bases de datos. El estándar ha evolucionado para incluir características orientadas a objetos, permitiendo que la gestión de objetos sea consistente y compatible entre distintos SGBD que sigan esta norma. Por ejemplo, SQL:1999 introduce UDTs (User-Defined Types), que permiten

definir objetos en la base de datos, y UDFs (User-Defined Functions) para manipular estos objetos de manera estándar.

Además de las características propias de los lenguajes relacionales, este lenguaje destacaba por haber sido desarrollado para **manejar objetos**. Como principal característica se puede destacar la posibilidad de **definir tipos de datos.**

Se trata de una revisión profunda que añade un número considerable de tipos poco convencionales, además de permitir también la definición de tipos compuestos o estructurados de datos.

Los principales tipos que incorpora son:

- **Boolean.** Hasta entonces, los valores lógicos se solían indicar usando el tipo BIT.

- **Grandes objetos**. SQL99 define dos tipos de objetos grandes, el llamado BLOB (Binary Large Object), adecuado para almacenar datos binarios como por ejemplo imágenes, videos, música, certificados digitales, etc. El otro tipo se denomina CLOB (Character Large Object), ideal para datos extensivos de tipos texto como por ejemplo informes, páginas web, artículos, etc.

- **Colecciones.** Permite almacenar de forma directa colecciones enteras de datos tanto de tipo básico como de tipo estructurado.

- **Tipos compuestos o estructurados**. Gracias a la incorporación de estos tipos de datos es posible crear tipo de datos definidos por el usuario.

- **Referencias a tipos estructurados**. Se trata de un tipo especial que actúa como apuntador de tipos compuestos. Son útiles porque permiten hacer una abstracción del lugar (mesa) donde realmente se almacenarán estos tipos. El sistema está preparado para realizar un almacenamiento por defecto, de forma que estos tipos son la única manera de referenciar los datos almacenados.

- **Permite la creación de métodos y funciones**. Dicho esto, y antes de continuar, es importante tener claro la diferencia entre ambos conceptos. Como resumen, se podría decir que los métodos son funciones con restricciones y aumentos.

EJEMPLO PRÁCTICO

En la empresa en la que trabajas necesitas almacenar imágenes de los productos en la base de datos. Tu jefe te pide que utilices el tipo de datos BLOB en Oracle para almacenar estas imágenes desde una aplicación en Java.

¿Cómo lo realizarías?

- Para resolver el problema, creas una tabla que incluye una columna de tipo BLOB y utilizas Java y JDBC para insertar las imágenes.

```
CREATE TABLE productos (
    id NUMBER PRIMARY KEY,
    nombre VARCHAR2(100),
    imagen BLOB
);
```

- Preparas el programa Java para insertar una imagen con PreparedStatement e importas las librerías necesarias:

```
String sql = "INSERT INTO productos (id, nombre, imagen) VALUES
(?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1, 1);
pstmt.setString(2, "Camiseta Deportiva");

File imagen = new File("camiseta.jpg");
FileInputStream fis = new FileInputStream(imagen);
pstmt.setBinaryStream(3, fis, (int) imagen.length());

pstmt.executeUpdate();
```

1.3 La interfaz de programación de aplicaciones de la base de datos

Una **interfaz de programación de aplicaciones**, también llamada **API**, es un conjunto de funciones que facilitan el intercambio de mensajes o datos entre dos aplicaciones. Facilita que dos aplicaciones que están trabajando al mismo tiempo se comuniquen e intercambien información.

El funcionamiento es muy sencillo, la API toma la información y realiza todo el trabajo de comunicación de forma transparente para la aplicación. Las APIs permiten:

- Acceder a bases de datos.
- Realizar la comunicación cliente servidor.
- Realizar una comunicación en tiempo real.
- Realizar una comunicación peer to peer.

Además de estas funciones, también permiten recuperar errores, traducir datos, seguridad, manejo de colas y todo esto con una interfaz fácil de aprender.

Durante el módulo y en esta misma unidad ya se han estudiado algunas APIS utilizadas en Middleware como, por ejemplo:

- API JDBC
- API ODBC

PARA SABER MÁS

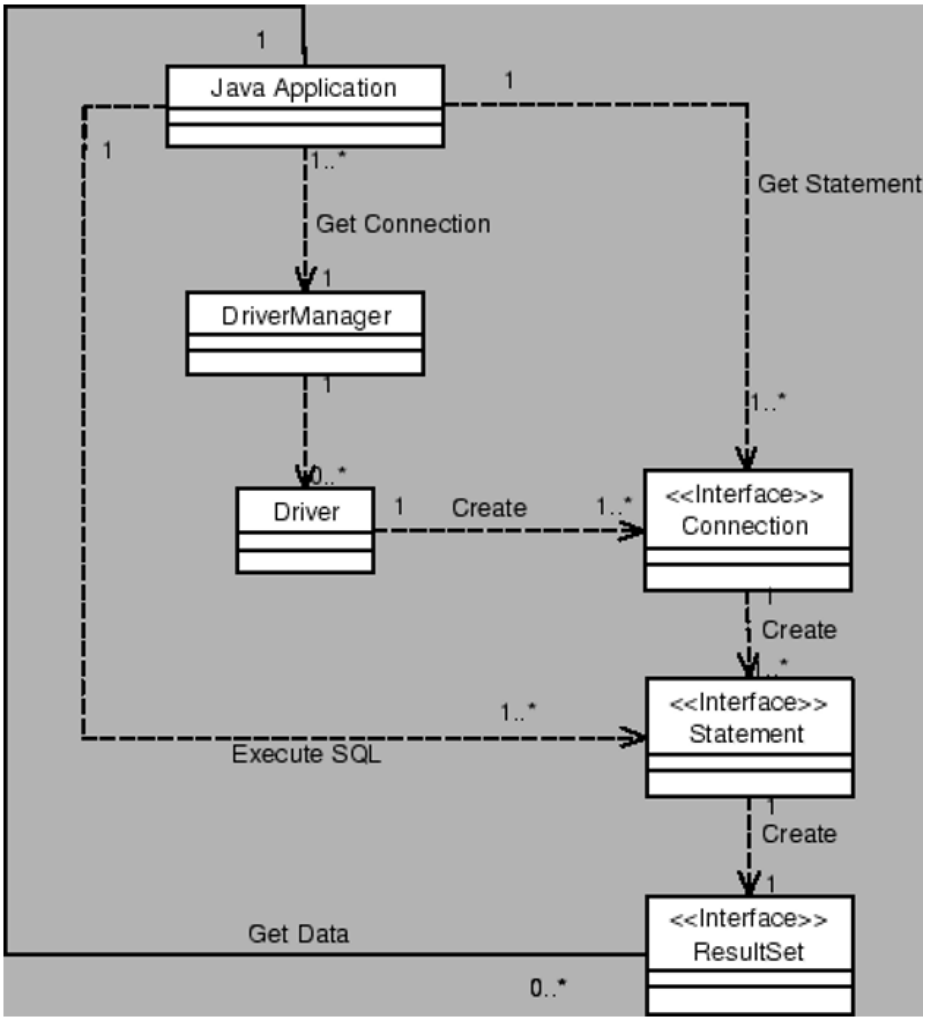
En la web de Oracle puedes encontrar documentación oficial sobre las diferentes APIs.

1.3.1 Acceso a las funciones del gestor desde el lenguaje de programación

Existen alternativas al uso de estos protocolos que permiten acceder a distintas bases de datos tan solo cambiando, por ejemplo, en el caso de la Plataforma Java, el driver JDBC por el que ofrezca el proveedor del servicio al que se quiera acceder.

JDBC es la API que permite la conexión con las bases de datos utilizando el lenguaje de programación Java, independientemente del sistema operativo que se utilice y de la base de datos a la cual se quiera acceder. Para esto, es necesario que el proveedor ofrezca un driver JDBC que cumpla con dicha especificación y traduzca las llamadas JDBC

a las APIs correspondientes. En la siguiente imagen se puede ver la arquitectura general de un driver JDBC.



API JDBC

Fuente: Elaboración propia

PARA SABER MÁS

En la web de Oracle se puede encontrar documentación oficial sobre la API JDBC:

RECUERDA

En anteriores unidades se dedicó código y estudio a los diferentes conectores dentro de Java para poder realizar las conexiones contra los diferentes tipos de bases de datos, así como al desarrollo de conexiones y múltiples acciones con una base de datos relacional.

1.3.2 Acceso y uso de tipos especiales

La incorporación de los tipos de datos BLOB, CLOB o ARRAYS obligó a rediseñar las clases de obtención de datos, puesto que pueden representar objetos de grandes dimensiones y puede llegar a no ser aconsejable trasladar toda la información desde el origen de datos a la máquina cliente.

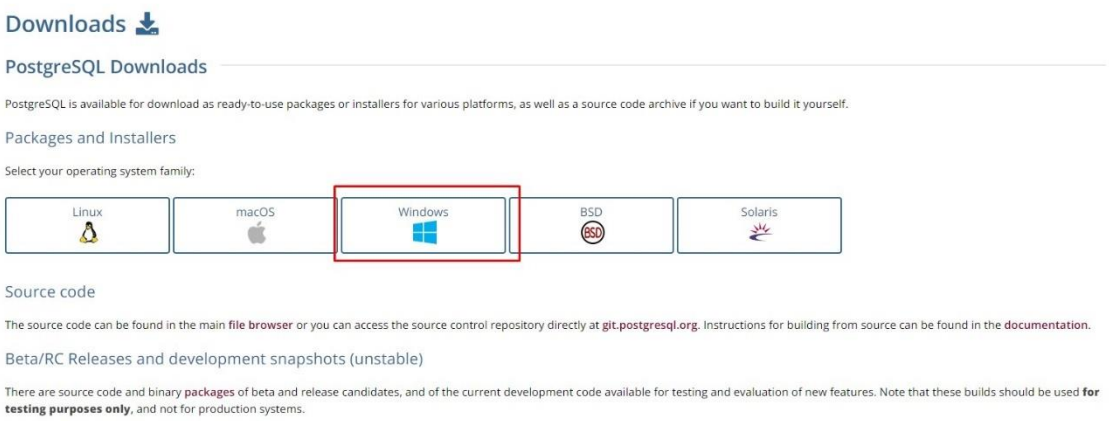
La manipulación de estos tipos de datos no se realiza volcándolas a tipos de datos estándares, sino que se utilizan unas clases especiales (java.sql.Blob, java.sql.Clob o java.sql.Array) que actúan como punteros lógicos a la información deseada a la base de datos.

Si usamos objetos Blob o Clob dispondremos del método getBinaryStream o getAsciiStream, respectivamente, para ir obteniendo la información parcialmente, a medida que la necesitamos. Los métodos devuelven un objeto InputStream que trataremos de forma habitual. Generalmente, dispondremos de una herramienta de manipulación de la información (procesador de textos, editores de imágenes, videos, gestor de ficheros, etc.) que trabaje con 'Streams'.

Si usamos objetos array, disponemos del método getArray para obtener un vector de Java. Es necesario saber que los datos no se encuentran a la memoria local, sino al SGBD. Este vector actúa a todos los efectos como si se tratara de un vector local, de forma que en la hora de codificar no notaremos la diferencia.

1.4 Configuración de un Proyecto con PostgreSQL

El primer paso es instalar PostgreSQL en el entorno de desarrollo. PostgreSQL que se puede descargar desde su página oficial, lo descargaremos para nuestro sistema operativo.



Descarga PostgreSQL
Fuente: Elaboración propia

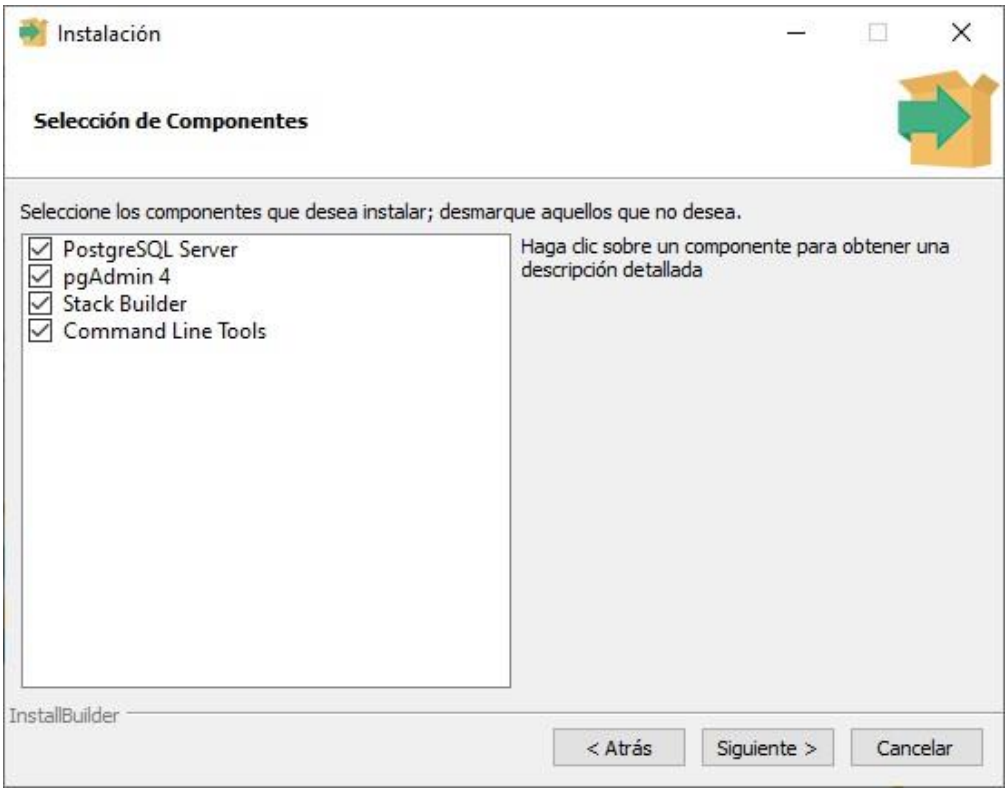
ENLACE DE INTERÉS

En esta web podrás descargar PostgreSQL dependiendo del SO:

PARA SABER MÁS

Aquí podrás descargar directamente PostgreSQL:

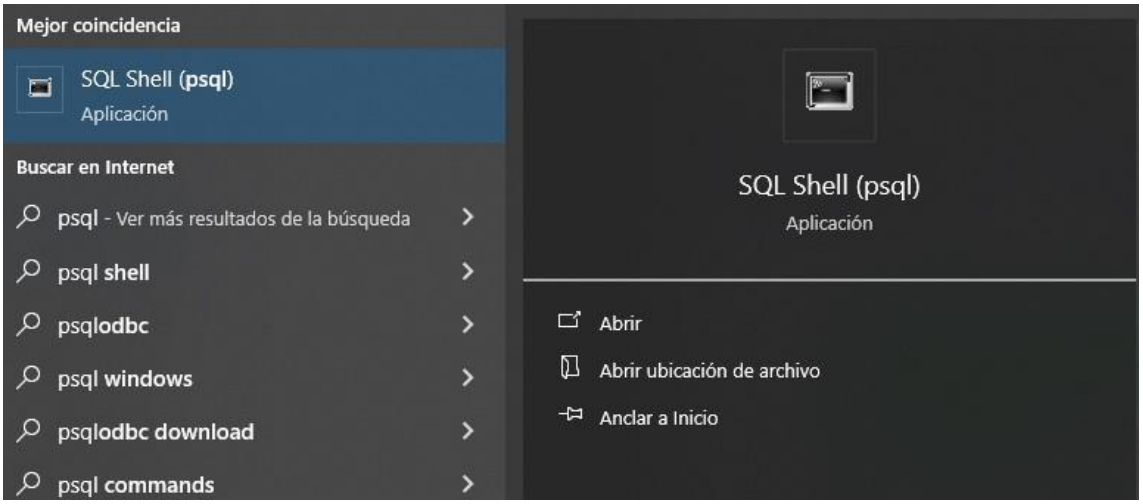
A continuación, vamos a instalar el SGBD. Para ello seleccionaremos todos los productos.



Productos PostgreSQL
Fuente: Elaboración propia

En contraseña pondremos “prueba” ya que estamos probando. Si se implementara este sistema en producción, deberíamos establecer alguna propia que cumpla las restricciones de seguridad, en puerto dejaremos el por defecto: 5432 y, en configuración regional, haremos lo mismo. Al terminar, se debe desactivar la instalación de stack builder. Una vez hayamos instalado PostgreSQL debemos reiniciar el ordenador

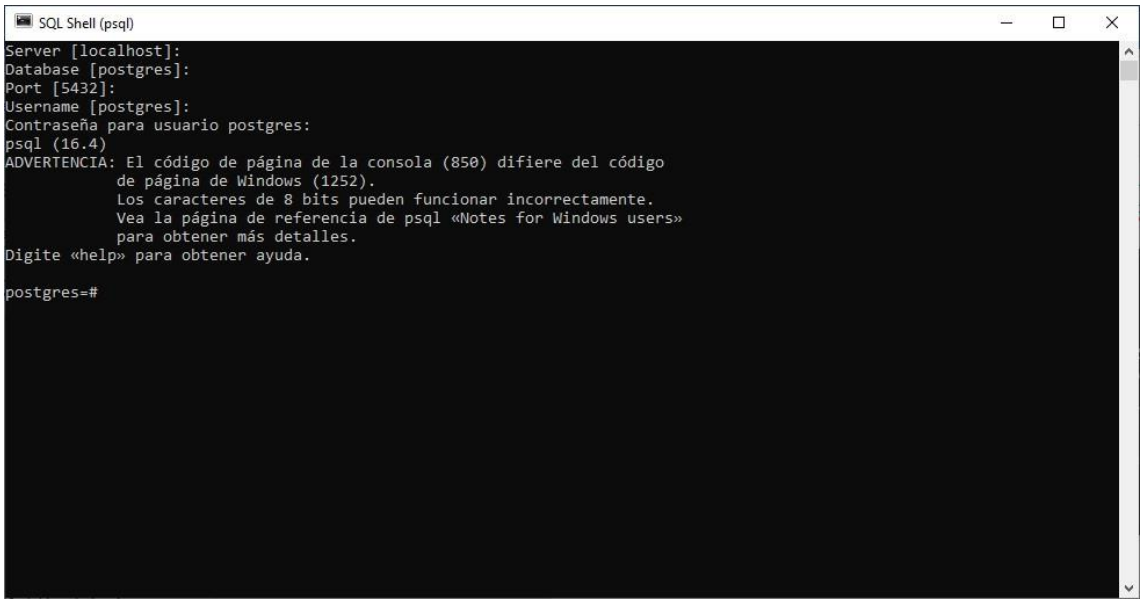
Para comprobar que todo esté bien, en el navegador de Windows ponemos psql (de PostgreSQL). Si nos aparece la consola de Postgre todo está bien, acto seguido la abrimos.



Consola psql

Fuente: Elaboración propia

Ahora al abrirla pulsaremos enter (para dejar los valores por defecto) hasta la contraseña: prueba. Luego veremos para lanzar los comandos (postgres=#).

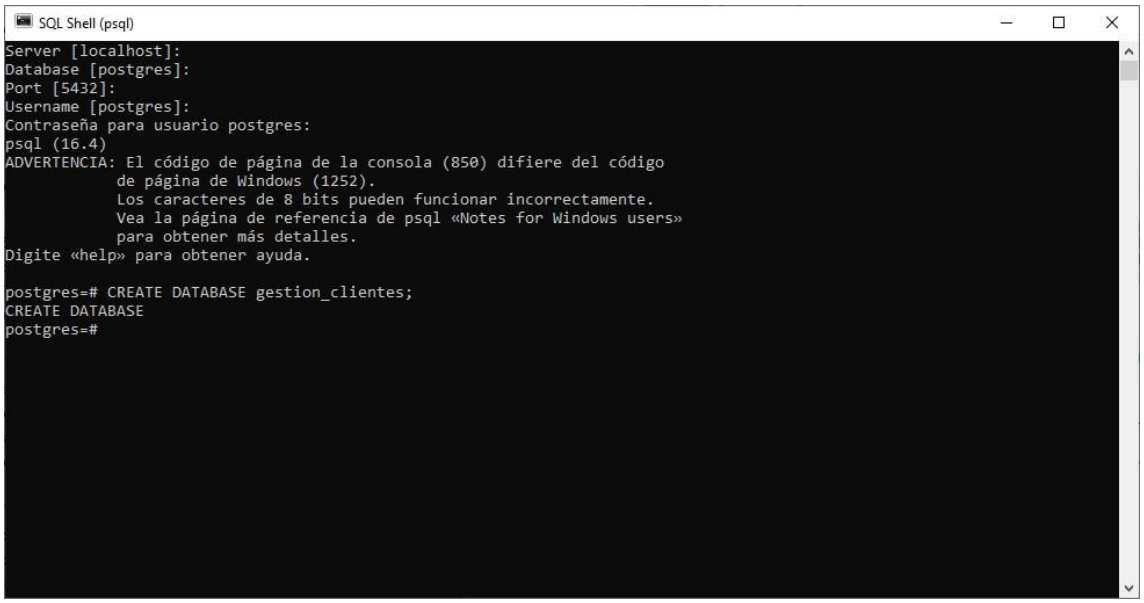


Lanzamiento psql

Fuente: Elaboración propia

Crearemos una base de datos para unos clientes con el lenguaje SQL, por lo que se debe introducir lo siguiente:

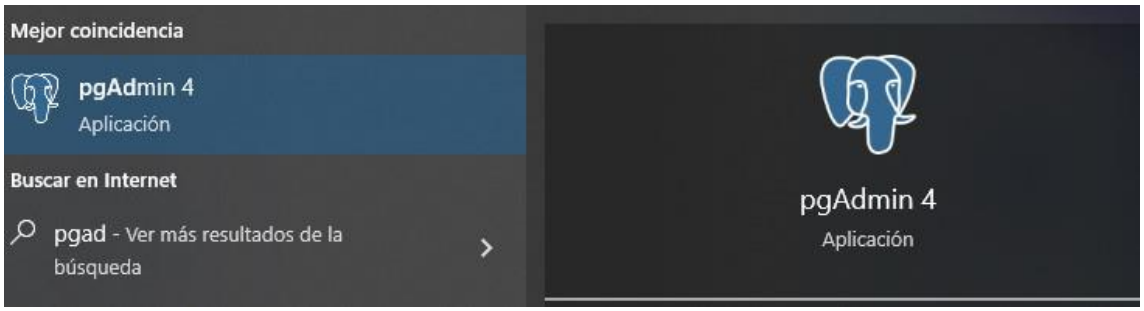
```
CREATE DATABASE gestion_clientes;
```



Creación BBDD

Fuente: Elaboración propia

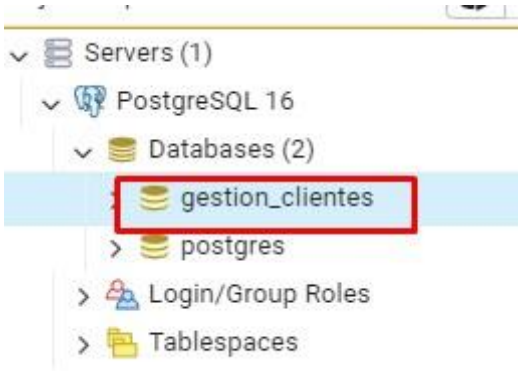
Para la inserción veremos pgadmin, que se trata de una interfaz gráfica que nos simplifica el proceso. Buscaremos pgadmin en nuestro navegador de Windows.



pgadmin

Fuente: Elaboración propia

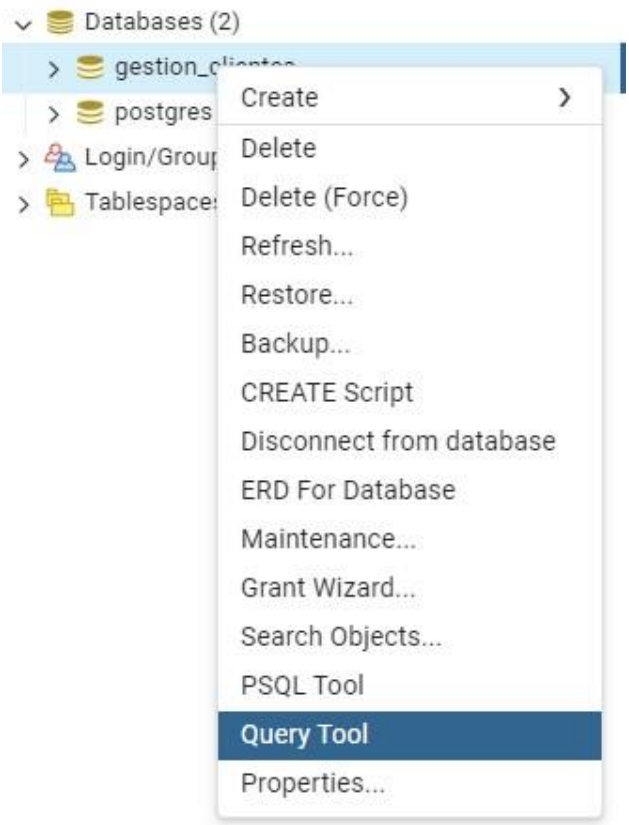
Introducimos la contraseña y veremos nuestra nueva BBDD.



BBDD pgadmin

Fuente: Elaboración propia

Ahora lanzaremos la consulta de creación de las tablas pulsando en el botón derecho y "Query Tool"



Lanzamiento de consulta
Fuente: Elaboración propia

En query escribiremos:

```
CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    correo_electronico VARCHAR(100),
    telefono VARCHAR(20)[],
    direccion VARCHAR(200)
);
```

Acto seguido ejecutamos la consulta pulsando en “Execute Query”

Execute Query
Fuente: Elaboración propia

Ahora insertaremos los datos con la siguiente consulta:

```
INSERT INTO clientes (nombre, correo_electronico, telefono,
direccion) VALUES
('Juan Pérez', 'juan.perez@email.com', ARRAY['+34 612 345 678',
'+34 623 456 789'], 'Calle Falsa 123, Ciudad Ejemplo'),
('María López', 'maria.lopez@email.com', ARRAY['+34 634 567 890'],
'Avenida Siempre Viva 742, Ciudad Ejemplo'),
('Carlos Sánchez', 'carlos.sanchez@email.com', ARRAY['+34 645 678
901', '+34 656 789 012'], 'Calle de la Rosa 10, Ciudad Ejemplo'),
('Ana Gómez', 'ana.gomez@email.com', ARRAY['+34 667 890 123'],
'Boulevard de los Sueños 101, Ciudad Ejemplo'),
('Luis Fernández', 'luis.fernandez@email.com', ARRAY['+34 678 901
234', '+34 689 012 345'], 'Paseo de las Flores 33, Ciudad
Ejemplo');
```

Si hemos hecho todo bien y ejecutamos “SELECT * FROM clientes;” deberíamos obtener el listado de nuestros clientes.

	id [PK] integer	nombre character varying (100)	correo_electronico character varying (100)	telefono character varying[] (20)	direccion character varying (200)
1	1	Juan Pérez	juan.perez@email.com	{"+34 612 345 678","+34 623 456 789"}	Calle Falsa 123, Ciudad Ejemplo
2	2	María López	maria.lopez@email.com	{"+34 634 567 890"}	Avenida Siempre Viva 742, Ciudad Ejemplo
3	3	Carlos Sánchez	carlos.sanchez@email.co...	{"+34 645 678 901","+34 656 789 012"}	Calle de la Rosa 10, Ciudad Ejemplo
4	4	Ana Gómez	ana.gomez@email.com	{"+34 667 890 123"}	Boulevard de los Sueños 101, Ciudad Ejemplo
5	5	Luis Fernández	luis.fernandez@email.com	{"+34 678 901 234","+34 689 012 345"}	Paseo de las Flores 33, Ciudad Ejemplo

Listado Clientes
Fuente: Elaboración propia

Ahora procedemos a la parte del proyecto de Java. Para ello crearemos un proyecto Maven donde agregaremos las siguientes dependencias.

```
<dependencies>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.3.1</version>
    </dependency>
</dependencies>
```

Una vez importadas las dependencias agregaremos una clase para realizar la conexión con el usuario y contraseña que creamos previamente.

```
package clientes;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexion {

    private static final String dbUrl =
"jdbc:postgresql://localhost:5432/gestion_clientes";
    private static final String dbUser = "postgres";
    private static final String dbPassword = "prueba";

    public static Connection getConnection() throws SQLException
{
        return DriverManager.getConnection(dbUrl, dbUser,
dbPassword);
    }
}
```

Ahora crearemos una clase para la inserción/busqueda de Clientes.

```
package clientes;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Array;

public class ClienteDAO {
    public void insertarCliente(String nombre, String correo,
String[] telefonos, String direccion) {
        String sql = "INSERT INTO clientes (nombre,
correo_electronico, telefono, direccion) VALUES (?, ?, ?, ?)";

        try (Connection conn = Conexion.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql))
{

            pstmt.setString(1, nombre);
            pstmt.setString(2, correo);

            Array arrayTelefonos = conn.createArrayOf("VARCHAR",
telefonos);
            pstmt.setArray(3, arrayTelefonos);

            pstmt.setString(4, direccion);

            pstmt.executeUpdate();
            System.out.println("Cliente insertado
correctamente.");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void listarClientes() {
        String sql = "SELECT * FROM clientes";

        try (Connection conn = Conexion.getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {

            while (rs.next()) {
                int id = rs.getInt("id");
                String nombre = rs.getString("nombre");
                String correo =
rs.getString("correo_electronico");
                Array telefonoArray = rs.getArray("telefono");
                String[] telefonos = (String[])
telefonoArray.getArray();
                String direccion = rs.getString("direccion");
```

```
                System.out.println("ID: " + id);
                System.out.println("Nombre: " + nombre);
                System.out.println("Correo: " + correo);
                System.out.println("Teléfonos: " + String.join(",
", telefonos));
                System.out.println("Dirección: " + direccion);
                System.out.println("---------------------------");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Por último, haremos la clase principal para interactuar con el SGBD.

```
package clientes;

public class Principal {

    public static void main(String[] args) {
        ClienteDAO clienteDAO = new ClienteDAO();

        // Insertar clientes
        clienteDAO.insertarCliente("Roberto Martínez",
"roberto.martinez@email.com", new String[]{"+34 655 234 567",
"+34 666 345 678"}, "Calle del Sol 456, Ciudad Ejemplo");
        clienteDAO.insertarCliente("Elena Rodríguez",
"elena.rodriguez@email.com", new String[]{"+34 677 456 789"},
"Avenida del Mar 789, Ciudad Ejemplo");

        // Listar clientes
        clienteDAO.listarClientes();
    }
}
```

Vemos que nos aparecen los clientes que agregamos antes a la BBDD más los 2 que hemos agregado al código.

```
<terminated> Principal (2) [Java Application] C:\Users\Patricio\Downloads\c
Cliente insertado correctamente.
Cliente insertado correctamente.
ID: 1
Nombre: Juan Pérez
Correo: juan.perez@email.com
Teléfonos: +34 612 345 678, +34 623 456 789
Dirección: Calle Falsa 123, Ciudad Ejemplo
----------------------------
ID: 2
Nombre: María López
Correo: maria.lopez@email.com
Teléfonos: +34 634 567 890
Dirección: Avenida Siempre Viva 742, Ciudad Ejemplo
----------------------------
ID: 3
Nombre: Carlos Sánchez
Correo: carlos.sanchez@email.com
Teléfonos: +34 645 678 901, +34 656 789 012
Dirección: Calle de la Rosa 10, Ciudad Ejemplo
----------------------------
ID: 4
Nombre: Ana Gómez
Correo: ana.gomez@email.com
Teléfonos: +34 667 890 123
Dirección: Boulevard de los Sueños 101, Ciudad Ejemplo
----------------------------
ID: 5
Nombre: Luis Fernández
Correo: luis.fernandez@email.com
Teléfonos: +34 678 901 234, +34 689 012 345
Dirección: Paseo de las Flores 33, Ciudad Ejemplo
----------------------------
ID: 6
Nombre: Roberto Martínez
Correo: roberto.martinez@email.com
Teléfonos: +34 655 234 567, +34 666 345 678
Dirección: Calle del Sol 456, Ciudad Ejemplo
----------------------------
ID: 7
Nombre: Elena Rodríguez
Correo: elena.rodriguez@email.com
Teléfonos: +34 677 456 789
Dirección: Avenida del Mar 789, Ciudad Ejemplo
----------------------------
```

Ejecución PostgreSQL

Fuente: Elaboración propia

1.5 Ventajas de los gestores de bases de datos objeto relacionales

Una vez hemos visto las características podemos concluir que los gestores de bases de datos objeto relacionales tiene capacidades que los hacen útiles en escenarios donde se requiere una mayor flexibilidad en la gestión de datos:

- **Flexibilidad en la definición de datos:** la posibilidad de crear tipos de datos personalizados permite que los sistemas reflejen con mayor precisión la lógica del negocio.

- **Compatibilidad con SQL estándar:** muchos SGBD objeto-relacionales mantienen compatibilidad hacia atrás con SQL estándar, facilitando la transición desde bases de datos relacionales tradicionales.

- **Capacidades extendidas:** estos sistemas permiten almacenar estructuras de datos complejas, como arrays y objetos, dentro de la base de datos, lo que simplifica la integración con aplicaciones orientadas a objetos.

EJEMPLO PRÁCTICO

En la empresa de estampado donde trabajas quieren comenzar probando el cambio de bases de datos usando PostgreSQL y Java introduciendo objetos dentro de la base de datos para la gestión de múltiples correos electrónicos dentro de la tabla de clientes.

¿Cómo realizarías esta gestión usando Arrays?

- Para resolver el problema necesitas poder cambiar o crear un nuevo método dentro de las sentencias DML que permita trabajar con estos arrays.

```
INSERT INTO CLIENT (id, correos_electronicos)
VALUES(1, ARRAY['aaa@a.es', 'bbb@bb.es', 'ccc@m.es']);
```

- Desde el punto de vista de la nueva función y Java se deberá realizar el siguiente cambio

```
PreparedStatement comStm = null;
...
StringBuilder comStr = new StringBuilder();
comStr.append("INSERT INTO client (id, correos_electronicos) ");
comStr.append("VALUES(?, ?)");
...
comStm = getConnexio().prepareStatement(comStr.toString());

comStm.setInteger(1, entidad.getId());
Array correo = getConnexio().createArrayOf("varchar",
entidad.getInformacionDeContacto().getArrayCorreosElectronicos())
;
comStm.setArray(2, correo);

comStm.executeUpdate();
```

…

- El método createArrayOf de la clase Connection permite crear un objeto java.sql.Array a partir de un vector Java. La instancia java.sql.Array se podrá pasar como parámetro de un PreparedStatment, el cual realizará la conversión al estándar SQL para poder hacer la inserción correctamente.
- En el caso de una actualización:

```
PreparedStatement comStm = null;
...
StringBuilder comStr = new StringBuilder();
comStr.append("UPDATE CLIENT SET correos_electronicos[1] = ? ");
comStr.append("WHERE id=?");
...
comStm = getConnexio().prepareStatement(comStr.toString());

comStm.seString(camp, entitat.getInformacionDeContacto()
.getArrayCorreosElectronicos()[0]);
comStm.setInteger(1, entitad.getId());

comStm.executeUpdate();
...
```

2. CARACTERÍSTICAS DE LAS BASES DE DATOS ORIENTADAS A OBJETOS

Tu jefe en la empresa de estampados necesita preparar otro entorno de pruebas además de PostgreSQL, y es el uso de una base de datos orientada al objeto.

Para ello almacenarás los objetos en una base de datos ObjectDB utilizando JPA, un estándar que te proporciona un marco para la persistencia de datos en aplicaciones orientadas a objetos.

La última alternativa es usar directamente una **base de datos orientada a objetos**. Estas bases de datos hacen persistentes los objetos de memoria, sin que haga falta ninguna transformación. Por lo tanto, se almacenan como objetos. Posteriormente también serán recuperados como objetos, también sin hacer ninguna transformación.

A pesar de que sin hacer un análisis muy profundo puede parecer que se trata de la mejor solución, esta modalidad de bases de datos presenta ciertas peculiaridades que hay que tener mucho en cuenta. Actualmente encontramos al mercado dos tipos de bases de datos orientadas a objeto.

Las que cumplen el estándar propuesto por el ODMG (Objeto Database Management Group) a finales del año 2000 y las llamadas bases de datos de tipos **NoSQL**, iniciativas tecnológicas posteriores. El lenguaje de consulta y manipulación de las primeras tiene

claras influencias de SQL. El de las segundas es próximo a algunos lenguajes de programación orientados a objetos (como Java, C#, C++ o Smalltalk).

El **ODMG** se deshizo en 2001, después de culminar el estándar llamado ODMG 3.0. Todo y el tiempo pasado, se trata de un estándar muy infrautilizado. Los lenguajes que especifica (ODL y OQL), a pesar de tener una potencia expresiva grande y permitir una adaptación real a la sintaxis habitualmente utilizada por la mayoría de los lenguajes orientados a objetos, presenta una enorme influencia del paradigma relacional.

ARTÍCULO DE INTERÉS

Conoce más sobre las diferentes alternativas de bases de datos NoSQL.

De hecho, ha representado una importante influencia en la especificación otros estándares, como JDO o JPA, que se postulan como puente entre sistemas gestores de bases de datos objeto-relacionales y las aplicaciones orientadas al objeto.

Sobre **los objetos almacenan el estado y su comportamiento**. Este comportamiento se define mediante operaciones sobre los datos. Una operación o función se especifica en dos partes:

- **Firma de la operación**: incluye el nombre y los tipos de datos de sus argumentos.
- **Implementación de la operación**: se especifica de forma separada y permite modificarse sin afectar a la signatura.

Los programas desarrollados permiten, por tanto, operar sobre los datos a través de llamadas a dichas operaciones mediante su firma; es decir, indicando sus nombres y los argumentos necesarios. Esto se haría de manera independiente a la forma en la que se haya implementado, por lo que se podría decir que se cumple la independencia entre programas y operaciones.

Al tratarse de bases de datos más evolucionadas, utilizan conceptos del paradigma orientado a objetos, entre los que cabe destacar:

- Identidad de objetos.
- Constructores de tipos.
- Encapsulamiento: permite ocultar la información al resto de los objetos impidiendo de esta manera accesos incorrectos o conflictos.
- Compatibilidad con los lenguajes de programación.
- Jerarquías de tipos.
- Manejo de objetos complejos.
- Polimorfismo: propiedad de una operación mediante la cual puede ser aplicada a distintos tipos de objetos.
- Sobrecarga de operadores.
- Creación de versiones.
- Herencia: permite que los objetos hereden comportamiento dentro de una jerarquía de clases.

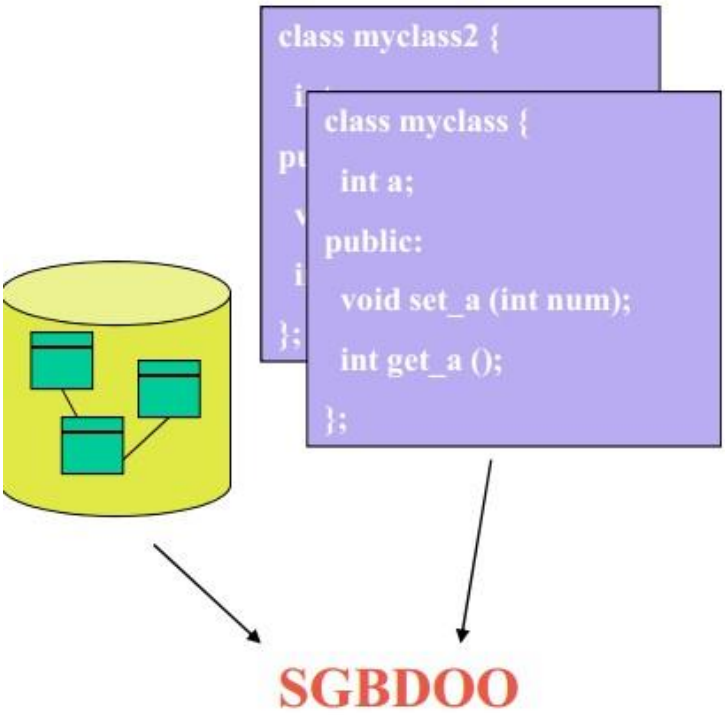
ARTÍCULO DE INTERÉS

Sistemas gestores de bases de datos orientadas a objetos.

2.1 Estructura de una BDOO

Los Sistemas de gestión de Bases de datos Orientados a objetos no están basados en sistemas relacionales como veíamos en el apartado anterior con los objeto-relacionales.

En la siguiente imagen se puede observar la estructura de una BDOO:



Esquema SGBDOO
Fuente: Elaboración propia

En las BDOO, la organización de los datos, lo que se denomina ODMG (Object Data Management Group), integra las capacidades de una BBDD y de la programación orientada a objetos, de forma que los datos de la BBDD aparezcan como objetos.

El **ODMG** define:

- Un modelo de objetos estándar para el diseño de estas BBDD.
- El lenguaje de definición de objetos o ODL (Object Definition Languaje).
- Lenguaje de consultas denominado OQL.

En la siguiente imagen podemos observar una tabla de **los estándares y productos** tanto de **BD** Objetos Relacionales y de **BDOO.**

BD OBJETO-RELACIONAL	BD OBJETOS PUROS
Estándar SQL: 1999 (SQL3), Melton 1999 SQL: 2003 , Melton 2003 Productos Oracle (a partir de la V8) POSTGRES Universal Server de INFORMIX.	Estándar ODMG-93, Cattell (1994), Cattell (1995) ODMG V.2.0, Cattell (1997) ODMG V.3.0, Cattell (2000) Productos ObjectStore de Object Design. Persistencia de objetos en C++ y Java. POET. Persistencia de objetos en C++ y Java. Gemstone de Servi Logia, Meier y Stone (1987). Persistencia de objetos Smalltalk, soporta también C++ y Java.

Comparación objeto-relacional orientación a objetos
Fuente: Elaboración propia

Para modelar una BDOO utilizaremos diagramas de clases y diagramas de objetos, principalmente usando diagramas UML. En las siguientes imágenes tenemos las representaciones o ejemplos de una clase con UML y de asociaciones entre clases.

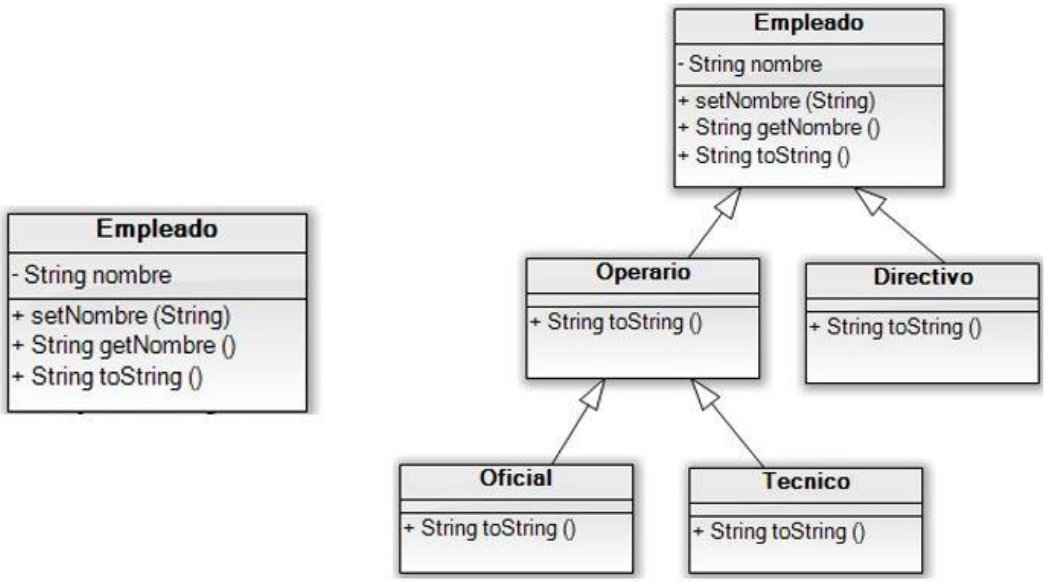


Diagrama UML de clases
Fuente: https://tinyurl.com/y35pbzsu

VÍDEO DE INTERÉS

Atiende a esta introducción a los diagramas UML.

2.2 Gestores de bases de datos orientadas a objetos

Son muchos los SGBDOO que se pueden encontrar en el mercado, por lo que en este apartado se citarán los más utilizados.

- ObjectStore
- Wakanda
- ObjectDB

2.2.1 ObjectStore

Esta BDOO propietaria, tal y como observamos en la web del fabricante que indicamos a continuación, es una base de datos orientada a proyectos con gran almacenamiento en cache. Está muy centrada en el rendimiento y el tratamiento de datos en tiempo real, para que las transacciones sean lo más rápido posibles.

La integración del almacenamiento está directamente relacionada con el lenguaje de programación usado como Java o C++, además de estar enfocada a una arquitectura descentralizada para una mejor escalabilidad de la implementación.

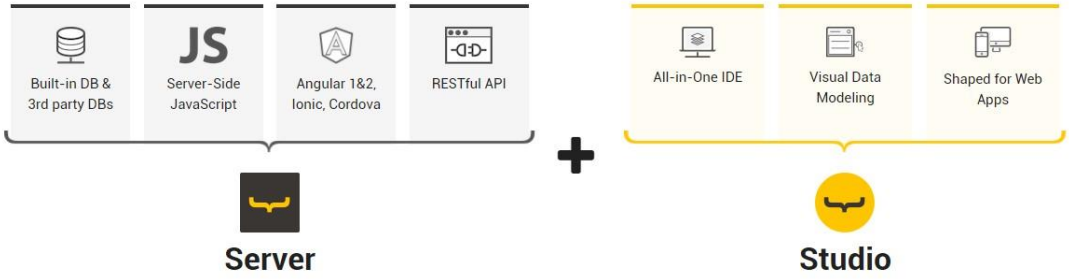
ENLACE DE INTERÉS

En la web del fabricante verás información sobre la base de datos:

2.2.2 Wakanda

Una solución opensource muy interesante realizada para proyectos JS, que proporciona una solución a todas las necesidades para el desarrollo de aplicaciones tal y como se ve en la siguiente imagen.

La base de datos Wakanda DB, es como podemos observar una base de datos integrada en la propia solución.



Pila de producto Wakanda
Fuente: https://tinyurl.com/y5v98ndv

PARA SABER MÁS

Aquí conocerás más información sobre las bases de datos orientadas a objetos, además de algunas marcas reconocidas, como Wakanda:

2.2.3 ObjectDB

ObjectDB es una base de datos orientada a objetos. Es software propietario, pero el fabricante, ObjectDB Software, ofrece una versión gratuita limitada que es suficiente para su estudio. Ofrece dos interfaces al programador: JPA (desde 2010) y JDO (desde su primera versión, el 2003).

La ventaja de utilizar estas interfaces es que se puede trasladar muy fácilmente programas realizados con mapeo objeto-relacionados a ObjectDB y al revés. Además, ObjectDB dispone también de un entorno gráfico para consultar y actualizar los datos. Este entorno gráfico admite la realización de querys con JPQL y JDOQL, que son los DML (Fecha Manipulation Language) respectivos de JPA y JDO.

PARA SABER MÁS

Aquí podrás visualizar la información ofrecida por el propio fabricante:

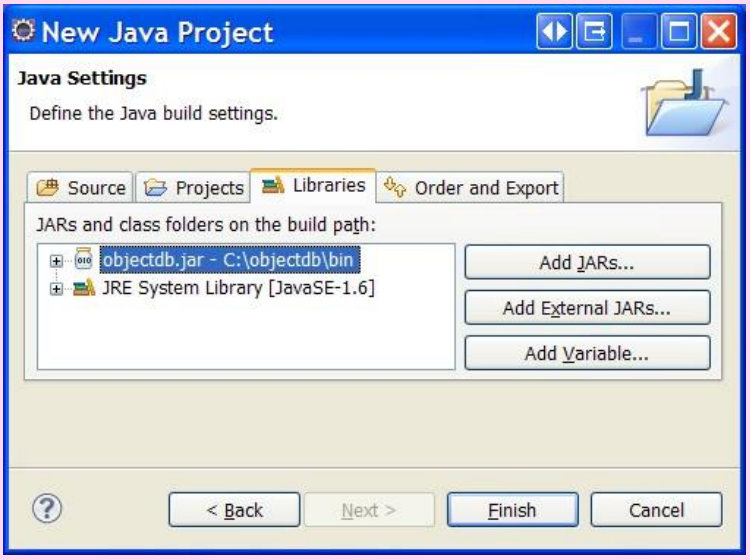
ENLACE DE INTERÉS

Accede para comprobar, y poder descargar, la última versión estable de ObjectDB:

EJEMPLO PRÁCTICO

Tu jefe dentro de la empresa de estampado se está planteando el uso de una base de datos orientada a objeto para tener un mejor mantenimiento y desarrollo de la aplicación. Para ello pretende utilizar ObjectDb como prueba para comprobar su funcionamiento.
¿Cómo realizaríamos un primer proyecto de prueba?

- El primer paso sería crear un nuevo proyecto y añadir la librería descargada del fabricante añadiendo el JAR.



Librería externa de ObjectDB
Fuente: https://tinyurl.com/yycufudj

- El siguiente paso sería crear tu primera entidad, en este caso la llamarás Point siguiendo el tutorial del fabricante.

```
import java.io.Serializable;
import javax.persistence.*;

@Entity
public class Point implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id @GeneratedValue
    private long id;

    private int x;
    private int y;

    public Point() {
    }

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Long getId() {
        return id;
    }

    public int getX() {
         return x;
    }

    public int getY() {
         return y;
    }

    @Override
    public String toString() {
        return String.format("(%d, %d)", this.x, this.y);
    }
}
```

- Crearías un Main para realizar la prueba de forma rápida.

```
package tutorial;

import javax.persistence.*;
import java.util.*;
public class Main {
    public static void main(String[] args) {
        // Open a database connection
        // (create a new database if it doesn´t exist yet):
        EntityManagerFactory emf =

Persistence.createEntityManagerFactory("$objectdb/db/p2.odb");
        EntityManager em = emf.createEntityManager();

        // Store 1000 Point objects in the database:
        em.getTransaction().begin();
        for (int i = 0; i < 1000; i++) {
            Point p = new Point(i, i);
            em.persist(p);
        }
```

```
        em.getTransaction().commit();

        // Find the number of Point objects in the database:
        Query q1 = em.createQuery("SELECT COUNT(p) FROM Point p");
        System.out.println("Total Points: " + q1.getSingleResult());

        // Find the average X value:
        Query q2 = em.createQuery("SELECT AVG(p.x) FROM Point p");
        System.out.println("Average X: " + q2.getSingleResult());

        // Retrieve all the Point objects from the database:
        TypedQuery<Point> query =
            em.createQuery("SELECT p FROM Point p", Point.class);
        List<Point> results = query.getResultList();
        for (Point p : results) {
            System.out.println(p);
        }

        // Close the database connection:
        em.close();
        emf.close();
    }
}
```

- Al ejecutar el ejemplo se crea un fichero con la base de datos. Para comprobar el correcto funcionamiento puedes usar el Explorador de ObjectDB para ver el resultado.

2.3 Lenguaje de consultas para objetos (OQL, Object Query Language)

Es necesario contar con un modelo que se encargue de especificar qué elementos se definirán y en qué manera se hará, para alcanzar la persistencia en las bases de datos orientadas a objetos. Este modelo representa un estándar y se denomina ODMG. Fue formado por un conjunto de representantes de la industria de las bases de datos con el fin de definir estándares para los SGBDOO.

La última versión de este estándar surgió en el año 2000 y se trataba de la versión **ODMG 3.0**, la cual está compuesta por:

- **Modelo de objeto:** permite que las implementaciones y diseños sean portables entre los distintos sistemas que lo soportan.

- **Lenguaje de definición de objetos (ODL):** lenguaje equivalente al lenguaje de definición de datos DDL de los SGBD tradicionales. Se encarga de definir los atributos y las relaciones entre tipos, así como de especificar la signatura de las operaciones. La sintaxis de este lenguaje extiende del lenguaje de definición de interfaces IDL perteneciente a la arquitectura CORBA.

- **Lenguaje de consulta de objetos (OQL):** lenguaje declarativo del estilo del lenguaje SQL que permite realizar las consultas sobre las BDOO de una manera eficiente. Para esto incluye primitivas de alto nivel. Está basado en el lenguaje SQL-92, por lo que su sintaxis es muy parecida a éste. Además, la sintaxis básica del lenguaje OQL sería la estructura SELECT como ocurre con SQL.

Por lo tanto, tendríamos un paralelismo entre los lenguajes que nos encontramos en los SGBD y los SGBDOO. En el siguiente diagrama podemos ver una representación de esas definiciones con ODL, siendo las principales características el no ser un lenguaje de programación completo y el ser independiente de cualquier lenguaje de programación.

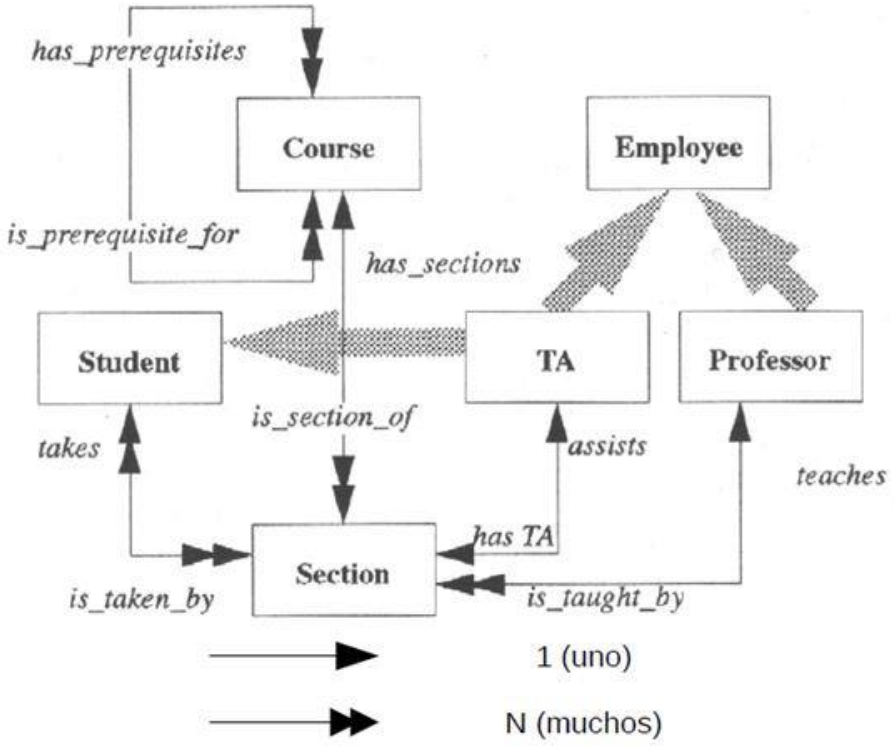


Diagrama ODL
Fuente https://dialnet.unirioja.es/descarga/articulo/5573952.pdf

Respecto al lenguaje de consultas OQL, podemos ver un ejemplo a continuación:

```
select distinct x.edad from x in Personas where x.nombre="Pedro"

select distinct struct(e : x.edad, s : x.sexo)
from x in Personas where x.nombre="Pedro"
```

Es un lenguaje orientado a objetos y declarativo. A pesar de no tener uso extendido actualmente, hay lenguajes propios de implementaciones de JPA como son Hibernate o Spring que tienen definiciones de lenguajes internos para las consultas con las bases de datos que se acercan mucho a las definiciones que acabamos de ver.

PARA SABER MÁS

Conoce más aspectos sobre la sintaxis de OQL:

ENLACE DE INTERÉS

Aquí podrás encontrar documentación muy extensa sobre HQL:

2.4 Gestión de la Persistencia y Transacciones. Consultas en Java con JPA

En Java la especificación JPA (Java Persistence API) proporciona un marco para la gestión de la persistencia de datos en aplicaciones orientadas a objetos, además es el más extendido en la actualidad. Cubriremos cómo JPA maneja las consultas y la persistencia de datos/objetos.

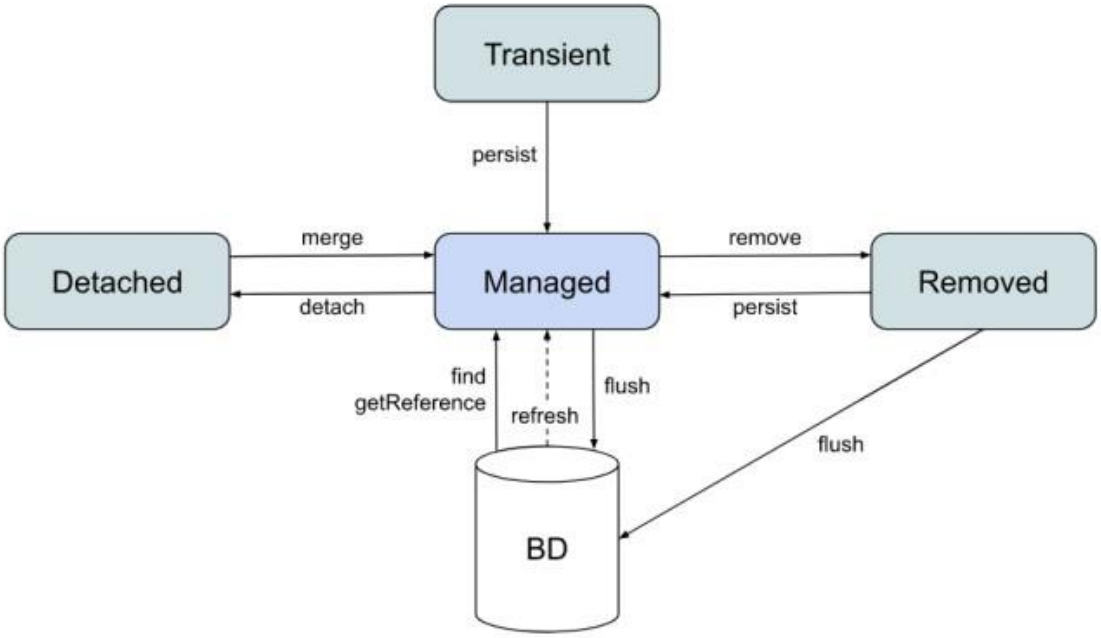
La persistencia de objetos en JPA permite que las entidades Java (objetos) se almacenen de manera duradera en la base de datos, asegurando que su estado persista más allá de la ejecución de la aplicación. Para conseguir la persistencia tenemos que crear un EntityManager. El EntityManager es el componente principal de JPA que gestiona el ciclo de vida de las entidades.

```
EntityManager em = emf.createEntityManager();
……..
em.close();
```

Para asegurar que todas las operaciones sobre las entidades se realicen dentro de una transacción, garantizando las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) debemos llamar al método commit.

```
em.getTransaction().begin();
em.getTransaction().commit();  // Aquí se confirman los cambios
```

En la unidad anterior estudiamos Hibernate, que implementa el estándar JPA, por lo que entender cómo funciona el ciclo de vida del objeto resulta similar a lo que estudiamos. A continuación, se muestra un diagrama con los estados de un objeto en una aplicación basada en JPA:



Estados en las sesiones.
Fuente: https://danielme.com/wp-content/uploads/2021/09/jpa-entity-lifecycle.png?w=656

Para almacenar objetos en la base de datos debemos hacer uso del método persist, la lectura se hace mediante el find. Si queremos actualizar una entidad debemos utilizar el merge y si queremos eliminarla podemos usar el método remove.

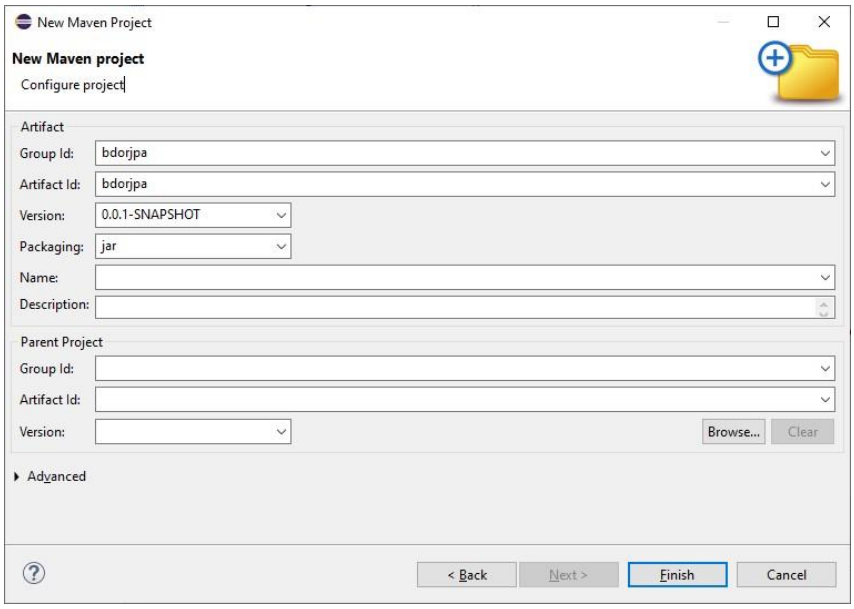
```
em.getTransaction().begin();

//Creación
Empleado nuevoEmpleado = new Empleado("Carlos", "Marketing",
60000);
em.persist(nuevoEmpleado);

//Lectura
Empleado empleado = em.find(Empleado.class, 1);

//Actualizacion
Empleado emp = em.find(Empleado.class, 1);
emp.setSalario(80000);
em.merge(emp);  // Actualizar la entidad en la base de datos

//Eliminacion
Empleado emp = em.find(Empleado.class, 1);
em.remove(emp);  // Eliminar el objeto de la base de datos

em.getTransaction().commit();  // Aquí se confirman los cambios
em.close();
```

PARA SABER MÁS

Visita esta web para conocer más aspectos sobre JPA:

2.5 Configuración de un Proyecto con ObjectDB

Para configurar el proyecto con ObjectDB vamos a utilizar el gestor de repositorios Maven, que vimos en la unidad anterior. Lo primero que tenemos que hacer es crear el proyecto.



Creación Maven

Fuente: Elaboración Propia

Después tenemos que añadir las dependencias correspondientes al pom.xml, en este caso al no estar en Maven central tenemos que agregarle el origen de los datos con la etiqueta repository.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>bdorjpa</groupId>
  <artifactId>bdorjpa</artifactId>
  <version>0.0.1-SNAPSHOT</version>

   <repositories>
        <repository>
            <id>objectdb</id>
            <name>ObjectDB Repository</name>
            <url>https://m2.objectdb.com</url>
        </repository>
    </repositories>

  <dependencies>
      <dependency>
            <groupId>com.objectdb</groupId>
            <artifactId>objectdb</artifactId>
            <version>2.8.1</version>
        </dependency>
</dependencies>

</project>
```

Una vez guardemos ya podemos empezar, vamos a empezar creando el modelo JPA (Persona.java).

```java
package bdorjpa;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Persona {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private int edad;

    // Constructor sin argumentos
    public Persona() {
    }
    // Constructor con argumentos
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "Persona{" +
                "id=" + id +
                ", nombre='" + nombre + '\'' +
                ", edad=" + edad +
                '}';
    }
}
```

Ahora haremos el fichero de configuración del JPA: persistence.xml, este fichero irá en resources en una carpeta que debemos crear llamada META-INF.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    version="2.1">

    <persistence-unit name="personasPU">
        <provider>com.objectdb.jpa.Provider</provider>
        <class>bdorjpa.Persona</class>
        <properties>
            <property name="javax.persistence.jdbc.url"
value="objectdb:personas.odb"/>
        </properties>
    </persistence-unit>
</persistence>
```

Por último, creamos la clase que interactúa con los datos según el estándar JPA.

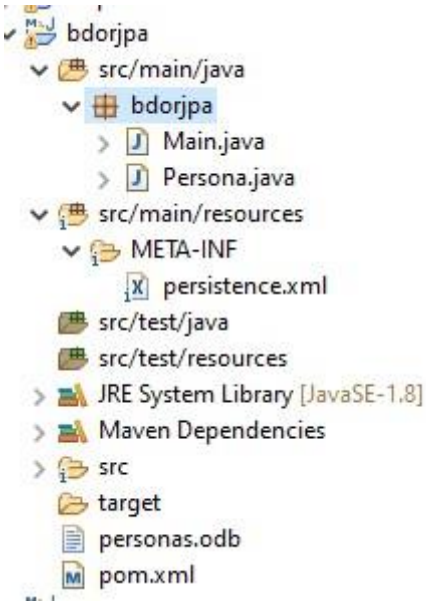
```
package bdorjpa;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        // Crear EntityManagerFactory y EntityManager
        EntityManagerFactory emf =
Persistence.createEntityManagerFactory("personasPU");
        EntityManager em = emf.createEntityManager();

        // Iniciar una transacción
        em.getTransaction().begin();

        // Crear una nueva persona y persistirla
        Persona persona1 = new Persona("Juan Pérez", 30);
        em.persist(persona1);

        Persona persona2 = new Persona("María López", 25);
        em.persist(persona2);

        // Confirmar la transacción
        em.getTransaction().commit();

        // Consultar todas las personas en la base de datos
        List<Persona> personas = em.createQuery("SELECT p FROM
Persona p", Persona.class).getResultList();
        for (Persona persona : personas) {
            System.out.println(persona);
        }

        // Cerrar EntityManager y EntityManagerFactory
        em.close();
        emf.close();
    }
}
```

Si lo ejecutamos este será el resultado.

```
Persona{id=1, nombre='Juan Pérez', edad=30}
Persona{id=2, nombre='María López', edad=25}
```

Ejecución ObjectDB
Fuente: Elaboración Propia

Al final la base de datos se almacenará en el fichero personas.odb y la estructura del proyecto debe quedar así.



Estructura Proyecto
Fuente: Elaboración Propia

ENLACE DE INTERÉS

Accediendo encontrarás un manual en el que podrás encontrar documentación muy interesante sobre JPA y ObjectDB.

EJEMPLO PRÁCTICO

En la empresa de software donde trabajas, han decidido implementar una nueva funcionalidad para gestionar múltiples domicilios asociados a cada cliente utilizando ObjectDB junto con Java implementando el estándar JPA. Cada cliente puede tener varias direcciones de domicilio, y estas deben ser almacenadas y gestionadas eficientemente en la base de datos. Además, tu jefe te solicita no crear fichero de mapeo XML para simplificar la arquitectura.

- Primero crearás el proyecto Maven llamándolo clientedom.

- Creas los modelos JPA (Cliente y Domicilio) y, para ser más estructurado, lo meterás en un paquete llamado modelos.

```
package modelos;
import javax.persistence.*;
import java.util.List;
@Entity
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Domicilio> domicilios;

    // Getters y Setters

      ……
}
```

```
package modelos;
import javax.persistence.*;
@Entity
public class Domicilio {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String calle;
    private String numero;
    private String ciudad;
    private String codigoPostal;
    private String tipo;

    // Getters y Setters

…
}
```

Como tu jefe te solicitó no tener fichero de mapeos, ya solo te quedaría la clase principal

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;
import javax.persistence.TypedQuery;

import modelos.Cliente;
import modelos.Domicilio;

import java.util.ArrayList;
import java.util.List;

public class Main {
```

```
    private static EntityManagerFactory emf =
Persistence.createEntityManagerFactory("objectdb:db/test.odb");

    public static void main(String[] args) {
        EntityManager em = emf.createEntityManager();

        // Guardar un cliente con domicilios
        guardarClienteConDomicilios(em, "Juan Perez",
crearDomiciliosEjemplo());

        // Consultar clientes por ciudad
        consultarClientesPorCiudad(em, "Ciudad A");

        em.close();
        emf.close();
    }

    public static void
guardarClienteConDomicilios(EntityManager em, String nombre,
List<Domicilio> domicilios) {
        em.getTransaction().begin();

        Cliente cliente = new Cliente();
        cliente.setNombre(nombre);
        cliente.setDomicilios(domicilios);

        em.persist(cliente);

        em.getTransaction().commit();

        System.out.println("Cliente guardado con éxito: " +
nombre);
    }

    public static void consultarClientesPorCiudad(EntityManager
em, String ciudad) {
        TypedQuery<Cliente> query = em.createQuery(
            "SELECT c FROM Cliente c JOIN c.domicilios d WHERE
d.ciudad = :ciudad", Cliente.class);
        query.setParameter("ciudad", ciudad);
        List<Cliente> clientes = query.getResultList();

        System.out.println("Clientes en la ciudad " + ciudad +
":");
        for (Cliente cliente : clientes) {
            System.out.println("Cliente: " +
cliente.getNombre());
        }
    }
```

```
            System.out.println("Cliente: " +
cliente.getNombre());
        }
    }

      private static List<Domicilio> crearDomiciliosEjemplo() {
        List<Domicilio> domicilios = new ArrayList<>();

        Domicilio domicilio1 = new Domicilio();
        domicilio1.setCalle("Calle 1");
        domicilio1.setNumero("123");
        domicilio1.setCiudad("Ciudad A");
        domicilio1.setCodigoPostal("12345");
        domicilio1.setTipo("Principal");

        Domicilio domicilio2 = new Domicilio();
        domicilio2.setCalle("Calle 2");
        domicilio2.setNumero("456");
        domicilio2.setCiudad("Ciudad B");
        domicilio2.setCodigoPostal("67890");
        domicilio2.setTipo("Secundario");

        domicilios.add(domicilio1);
        domicilios.add(domicilio2);

        return domicilios;
    }
}
```

Revisas el resultado de la ejecución.

```
Cliente guardado con éxito: Juan Perez
Clientes en la ciudad Ciudad A:
Cliente: Juan Perez
```

Ejecución Domicilios

Fuente: Elaboración Propia

2.6 Ventajas de las bases de datos orientadas a objetos

Como hemos hablado, las bases de datos orientadas a objetos son más flexibles cuando queremos trabajar con las características de la programación orientada a objetos, además de ser más eficientes, ya que no tenemos el desfase objeto relacional. Entre sus principales ventajas están:

- **Correspondencia directa con el modelo de objetos**: una de las mayores ventajas es la correspondencia directa entre el modelo de programación orientada a objetos y la estructura de la base de datos. Esto elimina la necesidad de mapeos complejos y reduce el riesgo de errores en la transformación de datos entre la aplicación y la base de datos.

- **Mayor flexibilidad y capacidad para manejar datos complejos**: las BDOO están diseñadas para manejar datos complejos de manera nativa, lo que permite a los desarrolladores trabajar con estructuras de datos ricas y multifacéticas sin necesidad de simplificarlas o transformarlas para adaptarse a un modelo relacional.

- **Reutilización y mantenimiento mejorados**: gracias a la utilización de herencia, polimorfismo y encapsulamiento, las BDOO permiten una mejor reutilización de código y facilitan el mantenimiento y la evolución de las aplicaciones.

- **Rendimiento en aplicaciones específicas:** en aplicaciones que requieren un manejo intensivo de objetos, las BDOO pueden ofrecer un rendimiento superior al evitar el desajuste objeto-relacional.

Sin embargo, también posee algunas desventajas:

- **Falta de estandarización:** a diferencia de las bases de datos relacionales que siguen estándares bien establecidos como SQL, las BDOO carecen de un estándar universalmente aceptado.

- **Dificultad en la integración con otras aplicaciones:** la falta de estandarización de las BDOO dificulta la integración con aplicaciones o sistemas que no estén basados en objetos.

- **Escalabilidad limitada:** las BDOO presentan problemas para escalar en aplicaciones que requieren gestionar grandes volúmenes de datos.

RESUMEN FINAL

Las bases de datos relacionales es uno de los modelos más usados tanto en pequeñas soluciones como en grandes aplicaciones. Sin embargo, existen otros modelos, como las bases de datos objeto-relacionales y orientadas a objetos cuyo principal objetivo es intentar disminuir el desfase objeto-relacional.

En el primer tipo, las SGBDOR, las mejoras incorporadas en el estándar SQL, los nuevos tipos orientados a manejar y almacenar objetos y sobre todo implementaciones como PostgreSQL u Oracle nos permiten dar el paso a una mejora de la implementación de nuestros datos sin abandonar el modelo relacional de bases de datos. Las ventajas de los SGBDOR incluyen la flexibilidad en la definición de datos personalizados, la compatibilidad con SQL estándar, y la capacidad de manejar estructuras de datos complejas, lo que simplifica la integración con aplicaciones orientadas a objetos.

En el segundo modelo, los SGBDOO, la implementación de los paradigmas de la Orientación a Objeto es total. Jerarquía de tipos, herencia, polimorfismo o sobrecarga de operadores son conceptos que hemos usado cuando programamos con lenguajes OO, y que por lo tanto son la base en el diseño de este tipo de bases de datos. Las bases de datos orientadas a objetos ofrecen ventajas como la correspondencia directa con el modelo de objetos, mayor flexibilidad para manejar datos complejos, y una mejor reutilización y mantenimiento del código. Además, en aplicaciones que requieren un manejo intensivo de objetos, las BDOO pueden ofrecer un rendimiento superior al evitar el desajuste objeto-relacional.

Algunas implementaciones como ObjectDB o Wakanda, nos permiten hacer pruebas de concepto para comprobar el paradigma de sGBDOO. JPA en Java proporciona un estándar para la gestión de la persistencia de datos en aplicaciones orientadas a objetos, facilitando la integración con bases de datos tanto objeto-relacionales como orientadas a objetos.